

Sistema de Control de Stock y Seguridad

Casas Lautaro, Farias Maira, Hoz Lucas y Massa Valentin
41744079, 42353622, 43920122 y 44510875
Lunes noche, L5

¡Universidad Nacional de La Matanza,
Departamento de Ingeniería e Investigaciones Tecnológicas,
Florencio Varela 1903 - San Justo, Argentina

Resumen. El presente trabajo describe el desarrollo de una solución IoT para el control de stock y seguridad utilizando un sistema embebido basado en ESP32 y una aplicación Android comunicados mediante WiFi. La lógica de funcionamiento se implementó mediante una Máquina de Estados Finitos (FSM), compuesta por los modos Embebido Virgen, Stock y Seguridad. En el modo Stock, el sistema monitorea la cantidad de elementos mediante celdas de carga y genera alertas cuando el nivel disponible desciende por debajo de un umbral establecido. En el modo Seguridad, detecta variaciones de peso que puedan indicar una manipulación no autorizada, activando alarmas visuales y sonoras. La aplicación Android permite enviar comandos al sistema embebido y visualizar la información recibida en tiempo real. Como resultado, se obtuvo un sistema funcional que integra monitoreo, comunicación inalámbrica e interacción con el usuario a través de una interfaz móvil.

Palabras claves: Internet de las Cosas (IoT), ESP32, Android, Máquina de Estados Finitos (FSM), Control de Stock, Seguridad.

1 Introducción

El sistema desarrollado tiene como objetivo brindar una solución IoT para el monitoreo y control de stock y seguridad mediante la integración de un sistema embebido basado en ESP32 y una aplicación Android comunicados a través de WiFi. La aplicación permite al usuario interactuar con el sistema de forma remota, enviando comandos para controlar los distintos modos de funcionamiento y visualizando en tiempo real la información obtenida por los sensores.

La lógica de funcionamiento se implementa mediante una Máquina de Estados Finitos (FSM), que organiza el comportamiento del sistema en tres estados: Embebido Virgen, Stock y Seguridad. El estado Embebido Virgen corresponde al modo inicial del sistema y representa el estado de espera cuando ninguno de los otros modos se encuentra activo. En el modo Stock, las celdas de carga permiten conocer la cantidad de elementos disponibles y generar alertas cuando el nivel de stock desciende por debajo de un umbral establecido, indicando la necesidad de reposición. En el modo Seguridad, el sistema detecta variaciones de peso que puedan representar una

manipulación de los elementos monitoreados, activando alarmas visuales y sonoras para notificar al usuario.

La aplicación Android complementa el funcionamiento del sistema embebido al ofrecer una interfaz sencilla para el envío de comandos y la visualización de la información recibida desde el ESP32, permitiendo una supervisión remota e intuitiva del estado del sistema.

2 Desarrollo

Repositorio GitHub:

<https://github.com/UNLAM-SOA/2026-SOA-Q1-L5>

Wokwi:

<https://wokwi.com/projects/463147445951076353>

Diagrama de Estados

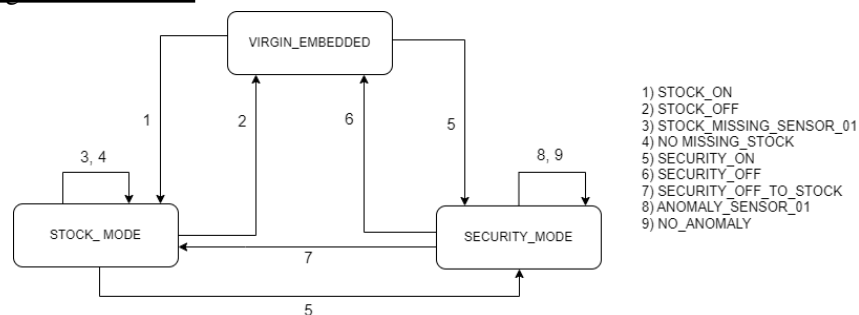


Diagrama del circuito en Wokwi

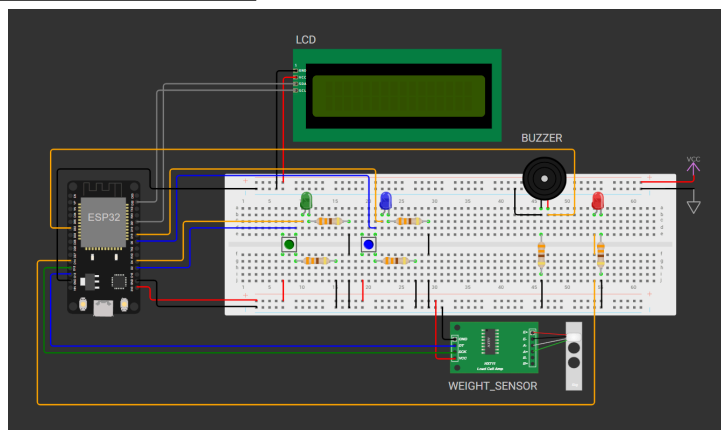
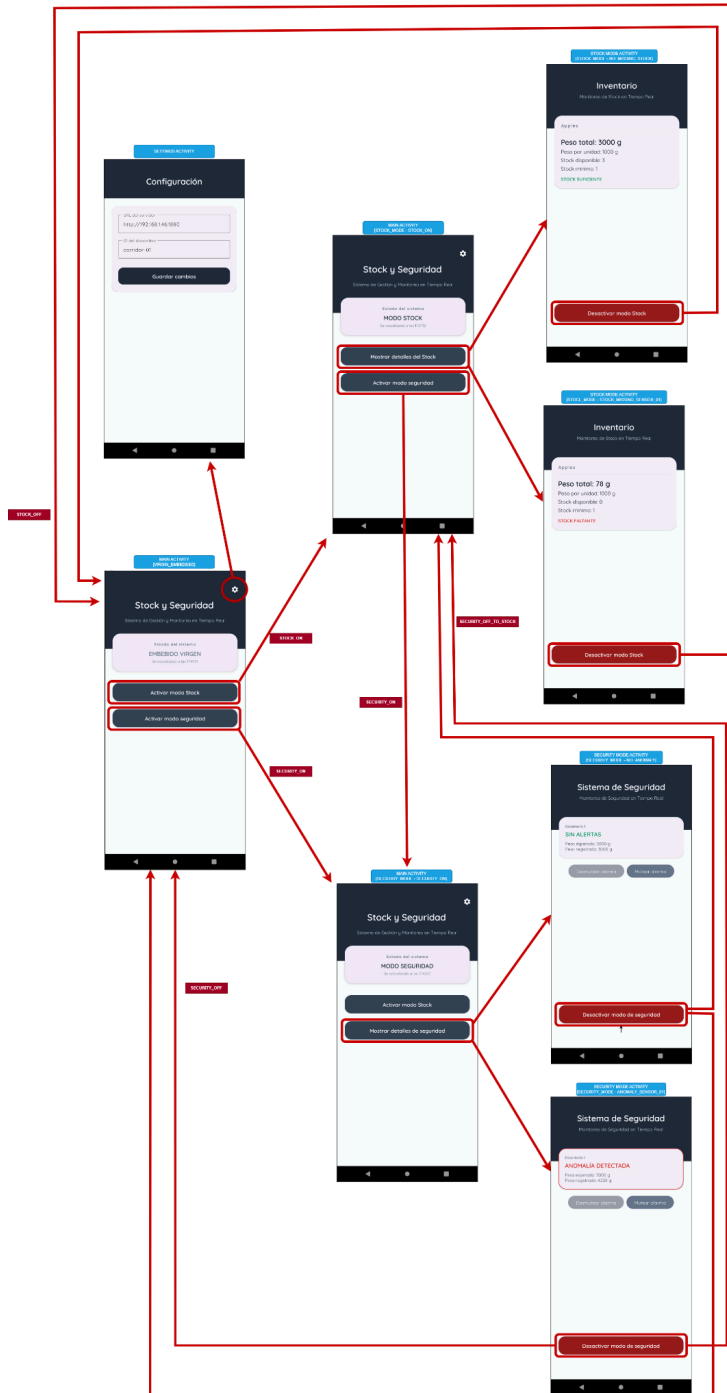


Diagrama de navegación de Android



Como se ilustra en el diagrama, la navegación de la aplicación se centra en la clase MainActivity, la cual actúa como el núcleo de la aplicación y adapta su interfaz dinámicamente según el estado del sistema ("Embebido Virgen", "Modo Stock" o "Modo Seguridad").

Desde esta Activity principal, el usuario puede acceder a las distintas pantallas de acuerdo con las acciones realizadas:

- SettingsActivity: Se accede mediante el ícono de configuración para establecer los parámetros de conexión con el servidor Node-RED (URL e ID del dispositivo). Al guardar la configuración o regresar a la pantalla anterior, el usuario vuelve a MainActivity.
- StockModeActivity: Al habilitar el Modo Stock (evento STOCK_ON), se accede a la pantalla de monitoreo de inventario. Allí se muestra el peso actual de cada góndola y el estado del stock, indicando si el nivel disponible es suficiente o si existe un faltante por debajo del umbral configurado. Al presionar el botón "Desactivar modo Stock", el sistema retorna a MainActivity, dejando deshabilitado dicho modo.
- SecurityModeActivity: Al habilitar el Modo Seguridad (evento SECURITY_ON), se accede al panel de monitoreo de seguridad. Esta pantalla muestra, para cada estantería, el peso de referencia registrado al activar el modo y el peso actual medido por el sensor, indicando si el sistema se encuentra en condiciones normales o si se detectó una anomalía. Además, permite mutear o desmutear de forma remota el buzzer del sistema embebido. Al presionar el botón "Desactivar modo Seguridad", el comportamiento depende del estado previo del sistema: si el Modo Stock se encontraba activo antes de habilitar la seguridad, la aplicación retorna a StockModeActivity para continuar el monitoreo del inventario; en caso contrario, regresa a MainActivity, quedando el sistema en el estado Embebido Virgen.

Manual de Usuario de la Aplicación Android

El presente manual detalla los procedimientos para operar el Sistema de Control de Stock y Seguridad a través de la aplicación móvil, así como la respuesta física esperada en el sistema embebido ante cada comando.

1. Configuración Inicial (Conexión)

Acción en la App: Al iniciar la aplicación por primera vez, el usuario debe dirigirse al ícono de configuración (engranaje) en la esquina superior derecha de la pantalla principal. Allí debe ingresar la URL del servidor Node-RED y el identificador (ID) del dispositivo físico a controlar, presionando luego "Guardar cambios".

Interacción con el Embebido: Esto establece el canal de comunicación entre la aplicación y el sistema embebido a través de Node-RED. A partir de este

momento, la aplicación comenzará a consultar periódicamente el estado del ESP32 para reflejar su información en tiempo real.

2. Estado de Reposo (Embebido Virgen)

Acción en la App: Si no hay ningún modo activado, la pantalla principal mostrará el estado "EMBEBIDO VIRGEN" junto con la hora de la última actualización recibida.

Interacción con el Embebido: Físicamente, el microcontrolador se encuentra a la espera de comandos. Los LEDs indicadores y el buzzer permanecen apagados, y la pantalla LCD del sistema mostrará únicamente la información básica del grupo (L5).

3. Operación en Modo Stock

Activar el modo: El usuario debe presionar el botón "Activar modo Stock" en la pantalla principal. Una vez activado, el estado del sistema pasará a indicar "MODO STOCK" y el botón cambiará a "Mostrar detalles del Stock". En el sistema físico, el LED verde se encenderá y la pantalla LCD pasará a mostrar el stock disponible calculado por la celda de carga.

Monitoreo: Al ingresar a "Mostrar detalles del Stock", el usuario puede visualizar en tiempo real el peso total y el stock disponible de cada estantería.

Alertas: Si al retirar elementos físicos de la estantería el peso desciende por debajo del mínimo configurado, el sistema embebido detectará el faltante. El LED rojo de la estantería afectada se encenderá y la pantalla LCD mostrará el mensaje "Stock missing on sensor!". Simultáneamente, la aplicación móvil actualizará su pantalla mostrando el estado "STOCK FALTANTE" en color rojo. Cuando el stock vuelve a estar por encima del mínimo, el LED rojo se apaga automáticamente y la app vuelve a mostrar "STOCK SUFICIENTE" en verde.

Desactivar el modo: Al presionar "Desactivar modo Stock" en la aplicación, se envía el comando de apagado al ESP32. El LED verde se apaga, la pantalla LCD se limpia y el sistema retorna al estado Embebido Virgen.

4. Operación en Modo Seguridad

Activar el modo: Al presionar "Activar modo seguridad" en la pantalla principal, el estado del sistema pasará a indicar "MODO SEGURIDAD". Físicamente, el LED azul se enciende y el sistema registra el peso actual de la celda de carga como peso esperado, que servirá como referencia para detectar variaciones.

Detección de anomalías: Al ingresar a "Mostrar detalles de seguridad", el usuario observa el peso esperado frente al peso registrado en tiempo real. Si en el entorno físico se retira o agrega peso sin autorización, la celda de carga registra la variación. El ESP32 hace sonar el buzzer inmediatamente y enciende el LED de la estantería afectada. Simultáneamente, la aplicación recibe la alerta, resalta la tarjeta de la estantería afectada con un borde rojo y muestra el mensaje "ANOMALÍA DETECTADA".

Gestión de la Alarma: Mediante los botones "Mutear alarma" y "Desmutear alarma" en la aplicación, el usuario puede silenciar o reactivar el buzzer físico de forma remota, sin necesidad de desactivar todo el sistema de seguridad.

Desactivar el modo: Al presionar "Desactivar modo de seguridad", el ESP32 apaga el LED azul, silencia el buzzer (si estaba sonando) y limpia las alertas en el LCD. Si el Modo Stock estaba activo previamente, el hardware y la aplicación retoman el monitoreo de inventario; de lo contrario, vuelven al estado Embebido Virgen.

3 Conclusiones

El desarrollo de este Trabajo Práctico Integrador permitió consolidar de manera exitosa un ecosistema IoT completo, logrando la intercomunicación fluida entre el hardware físico (ESP32 y sensores), un servidor middleware (Node-RED) y una interfaz móvil para el usuario final.

Recaudos tomados para que la aplicación sea tolerante a fallos

Para dotar al sistema de robustez, se implementaron medidas tanto a nivel de hardware como de software. En el sistema embebido, se aislaron las responsabilidades utilizando tareas de FreeRTOS; de esta forma, si el ESP32 pierde conexión y la tarea de red (xMQTTTask) entra en rutinas de reconexión automática, no se bloquea la ejecución de la Máquina de Estados (FSM) ni el muestreo de los sensores locales. Asimismo, se aplicó debouncing (antirrebote) por software mediante temporizadores no bloqueantes (millis()) en la lectura de los pulsadores para evitar falsas transiciones de estado. Por el lado de la aplicación Android, la tolerancia a fallos se logró utilizando programación defensiva en el parseo de datos (métodos optString, optJSONObject), lo que garantiza que si el hardware envía paquetes JSON incompletos o malformados, la aplicación no sufra un cierre abrupto (crash). Además, todas las peticiones de red se delegaron a hilos secundarios asincrónicos para mantener la interfaz gráfica (UI Thread) siempre responsiva.

Problemas encontrados y resueltos

Uno de los mayores desafíos durante la fase de integración física fue la inestabilidad en las lecturas de la celda de carga a través del módulo HX711. Inicialmente, el ruido

eléctrico y las mínimas vibraciones del entorno generaban fluctuaciones en el peso registrado, lo cual provocaba falsos positivos y disparaba el buzzer erróneamente en el Modo Seguridad. Este problema se resolvió programando una "ventana de tolerancia" lógica (ANOMALY_THRESHOLD) en el código; de esta manera, el sistema solo dispara la alarma si la variación de peso supera un margen de error preestablecido, filtrando el ruido de las mediciones. Otro inconveniente surgió al intentar procesar la lectura de sensores, actualizar el display LCD y mantener la conexión WiFi de forma simultánea en un único hilo de ejecución, lo que generaba retardos intolerables. La solución definitiva fue estructurar el código basándose en el paradigma de sistemas operativos en tiempo real (FreeRTOS) y colas de mensajes (xQueueSend), permitiendo la concurrencia cooperativa y asegurando que la FSM reaccione a los eventos sin sufrir pausas indeseadas.

Lecciones aprendidas

A nivel metodológico, el proyecto demostró la enorme ventaja de modelar el comportamiento del sistema utilizando una Máquina de Estados Finitos (FSM) antes de comenzar la etapa de codificación. Este enfoque abstracto permitió tener un control determinista sobre el hardware, facilitando el diseño y la depuración. Además, el trabajo dejó como lección la importancia de una arquitectura desacoplada y basada en middleware: al utilizar Node-RED como puente traductor entre protocolos (MQTT para el bajo consumo del ESP32, y HTTP REST para la aplicación Android), se logró que ambos extremos operaran de forma completamente independiente y asíncrona. Esto no solo simplifica el manejo de los estados, sino que sienta las bases para futuras expansiones del proyecto (como agregar más estanterías o clientes web) sin necesidad de modificar el núcleo del firmware existente.

4 Referencias